

ROS 2 커스텀 인터페이스

메시지 & 서비스 타입 정의

cchyun@gmail.com

Part 1. 커스텀 메시지 타입 정의

커스텀 메시지가 왜 필요한가?

- ROS 2 노드 간 데이터 교환 규격을 인터페이스(Interface)라고 함
 - 표준 메시지의 한계
 - `std_msgs`, `sensor_msgs` 등 표준 패키지가 제공되지만, 복잡한 데이터 묶음 전송은 어려움
 - 예: YOLO 탐지 결과(물체 이름, 정확도, 위치 좌표)를 `std_msgs/String` 하나로 표현 → **역부족**
 - 커스텀 메시지의 이점
 - 데이터의 구조화 → 코드 가독성 향상
 - 네트워크 통신 효율성 향상
 - 핵심 아이디어
 - 필요한 데이터 필드를 직접 정의 → 재사용 가능한 전용 메시지 타입 생성

.msg 파일 작성법 — 기본 타입 (Field Types)

- 커스텀 메시지는 `.msg` 확장자를 가진 텍스트 파일로 정의
 - 언어에 종속되지 않는 독립적인 필드 타입 사용

분류	타입	설명
논리형	<code>bool</code>	참/거짓
정수형	<code>int8</code> , <code>int16</code> , <code>int32</code> , <code>int64</code>	부호 있는 정수
정수형	<code>uint8</code> ~ <code>uint64</code>	부호 없는 정수
실수형	<code>float32</code> , <code>float64</code>	부동소수점
문자열	<code>string</code>	텍스트 데이터
바이트 배열	<code>byte[]</code>	바이트 묶음

- 중첩 메시지 타입 (Nested Types)

- 이미 정의된 다른 메시지를 필드로 사용 가능

```
geometry_msgs/Point position    # 다른 패키지의 메시지 포함
```

- 배열 필드 정의 (Arrays)

- 고정 크기 배열: `int32[10] data` (항상 10개의 정수)
- 가변 크기 배열: `int32[] data` (데이터 개수가 변할 수 있음)

- 상수(Constant) 정의

- 메시지 내에서 변하지 않는 값 정의
- 대문자로 작성하는 것이 관례

```
int32 ERROR_CODE=1
```

커스텀 메시지 패키지 구조

- 커스텀 메시지는 별도의 전용 인터페이스 패키지로 관리
 - 반드시 `ament_cmake` 빌드 유형이어야 함 (Python 전용 패키지 불가)

- 필수 디렉토리 구조

```
my_robot_interfaces/  
├── CMakeLists.txt  
├── package.xml  
└── msg/  
    ├── ObjectDetection.msg  
    └── ObjectDetectionArray.msg
```

- 패키지 생성 명령

```
ros2 pkg create --build-type ament_cmake my_robot_interfaces  
mkdir my_robot_interfaces/msg
```

- 메시지 생성을 위한 4가지 핵심 의존성

1. `ament_cmake` — 빌드 시스템 기본값
2. `rosidl_default_generators` — 인터페이스 파일을 Python/C++ 코드로 변환 (빌드 시 필요)
3. `rosidl_default_runtime` — 실행 시 메시지를 사용하기 위해 필요
4. `rosidl_interface_packages` — 이 그룹에 속해야 함을 명시

```
<buildtool_depend>ament_cmake</buildtool_depend>
<depend>std_msgs</depend>
<buildtool_depend>rosidl_default_generators</buildtool_depend>
<exec_depend>rosidl_default_runtime</exec_depend>
<member_of_group>rosidl_interface_packages</member_of_group>
```

- 예제 파일 정의 (`my_robot_interfaces` 패키지)

- `msg/ObjectDetection.msg`

```
string class_name      # 물체 클래스명 (예: 'person', 'dog')
float64 confidence     # 탐지 신뢰도 (0.0 ~ 1.0)
int32[4] bbox          # 바운딩 박스 좌표: [x_min, y_min, x_max, y_max]
```

- `msg/ObjectDetectionArray.msg`

```
std_msgs/Header header
ObjectDetection[] detections # 객체 배열
```


- CMakeLists.txt에서 메시지 생성을 지시

```
cmake_minimum_required(VERSION 3.8)
project(my_robot_interfaces)

find_package(ament_cmake REQUIRED)
find_package(std_msgs REQUIRED)
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/ObjectDetection.msg"
  "msg/ObjectDetectionArray.msg"
  DEPENDENCIES std_msgs
)

ament_package()
```

- 핵심 명령어

- `find_package(rosidl_default_generators REQUIRED)` — 인터페이스 생성 도구 찾기
- `rosidl_generate_interfaces(...)` — 빌드 대상 인터페이스 파일 등록

Python에서 커스텀 메시지 импорт 및 사용

```
import rclpy
from rclpy.node import Node
from my_robot_interfaces.msg import ObjectDetection, ObjectDetectionArray
from sensor_msgs.msg import Image
from ultralytics import YOLO
from cv_bridge import CvBridge

class YoloPublisher(Node):
    def __init__(self):
        super().__init__('yolo_publisher')
        self.bridge = CvBridge()

        # YOLOv8 모델 로드 (Nano = 가장 가벼움)
        self.model = YOLO('yolov8n.pt')

        # 구독 & 퍼블리시
        self.create_subscription(Image, 'image_raw', self.callback, 10)
        self.pub = self.create_publisher(ObjectDetectionArray, 'image_yolo', 10)
```

```
def callback(self, msg):
    frame = self.bridge.imgmsg_to_cv2(msg, 'bgr8')
    results = self.model(frame, verbose=False)[0]

    array_msg = ObjectDetectionArray()
    array_msg.header = msg.header

    for box in results.bboxes:
        detection = ObjectDetection()

        detection.class_name = results.names[int(box.cls)]
        detection.confidence = float(box.conf)
        detection.bbox = [int(v) for v in box.xywh[0].tolist()]

        array_msg.detections.append(detection)

    self.pub.publish(array_msg)

def main(args=None):
    rclpy.init(args=args)
    node = YoloPublisher()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

자주 발생하는 오류와 해결법

오류 현상	원인	해결 방법
<code>ImportError: cannot import name...</code>	빌드 후 <code>source</code> 를 수행하지 않았거나 패키지 이름 오타	새 터미널에서 <code>source install/setup.bash</code> 실행
빌드 시 <code>rosidl</code> 관련 에러	<code>CMakeLists.txt</code> 에 <code>rosidl_generate_interfaces</code> 누락	CMake 명령어가 정확히 포함되었는지 확인
패키지 인식 불가	인터페이스 패키지를 <code>ament_python</code> 으로 생성함	메시지 패키지는 반드시 <code>ament_cmake</code> 타입이어야 함
빌드 순서 문제	메시지 패키지를 빌드하기 전에 노드 패키지를 먼저 빌드함	메시지 패키지를 우선 빌드하거나 전체 빌드 수행

Part 2. 커스텀 서비스 타입 정의

- `.srv` 파일은 요청(Request)과 응답(Response) 구조를 정의
 - `---` (세 개의 대시) 구분자로 요청/응답 영역을 구분
 - 복합 타입 필드: 다른 패키지 또는 동일 패키지의 커스텀 메시지를 필드로 사용 가능
- 예제 파일 정의 (`my_robot_interfaces` 패키지)

`srv/SaveDetections.srv`

```
# Request: 탐지 결과 리스트
ObjectDetection[] detections
---
# Response: 결과
bool success          # 저장 성공 여부
string file_path      # 저장된 파일 경로
```

- 아키텍처 제약: 인터페이스 패키지는 반드시 `ament_cmake` 빌드 타입
 - Python 노드에서만 사용하더라도 인터페이스 생성 도구(`rosidl`)가 CMake 기반으로 작동
- `CMakeLists.txt` 설정

```
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/ObjectDetection.msg"
  "msg/ObjectDetectionArray.msg"
  "srv/SaveDetections.srv"
)
```

- `call_async()` 를 사용하는 것이 표준 관행 — 노드 차단(blocking) 방지

```
import rclpy
from rclpy.node import Node
from my_robot_interfaces.srv import SaveDetections

class DetectionSaveServer(Node):
    def __init__(self):
        super().__init__('detection_save_server')
        # 서비스 생성: 타입, 이름, 콜백 함수
        self.srv = self.create_service(
            SaveDetections, 'save_detections', self.save_callback
        )

    def save_callback(self, request, response):
        # request.detections는 ObjectDetection 메시지의 리스트
        self.get_logger().info(f'Received {len(request.detections)} objects')

        # 저장 로직 (예시)
        response.success = True
        response.file_path = '/tmp/detections.json'
        return response
```


- 대규모 프로젝트: 인터페이스를 별도의 전용 패키지로 관리 — Best Practice

- 순환 의존성 방지
- C++, Python 등 여러 언어로 작성된 노드들이 동일한 정의를 쉽게 공유

- 권장 패키지 구조

```
my_robot_interfaces/  
├── msg/  
│   ├── ObjectDetection.msg  
│   └── ObjectDetectionArray.msg  
├── srv/  
│   └── SaveDetections.srv  
├── CMakeLists.txt  
└── package.xml
```

- 명명 규칙: 패키지 이름에 `_interfaces` 접미사 사용 권장

Part 3. 실습

Timestamp 동기화 시각화 & Launch 파일 작성

미션1 - 동기화가 왜 필요한가?

- 문제 상황

- `image_raw` (카메라)와 `image_yolo` (YOLO 탐지 결과)는 별도의 노드에서 비동기적으로 발행
- 네트워크 지연, 처리 시간 차이로 인해 같은 프레임에 대한 메시지가 다른 시점에 도착

- 동기화하지 않을 경우의 문제

- YOLO 결과가 이전 프레임의 이미지에 겹쳐 그려짐 → 시각적 불일치
- 탐지 박스 위치와 실제 물체 위치가 어긋남

- 해결책: `message_filters ApproximateTimeSynchronizer`

- 두 토픽의 `header.stamp` 를 비교하여 시간적으로 가장 가까운 메시지 쌍을 매칭
- `slop` 파라미터로 허용 시간 차이 설정 (예: 0.1초 = 100ms)

미션1 - message_filters 동기화 구조

- 동기화 파이프라인 구조

구성 요소	역할
<code>message_filters.Subscriber</code>	일반 구독자를 message_filters 체인에 연결
<code>ApproximateTimeSynchronizer</code>	두 토픽의 timestamp를 비교해 가장 가까운 쌍 매칭
<code>sync.registerCallback()</code>	매칭된 메시지 쌍을 하나의 콜백으로 전달

- ApproximateTimeSynchronizer 파라미터

```
ApproximateTimeSynchronizer(  
    [sub_image, sub_yolo],    # 동기화할 Subscriber 리스트  
    queue_size=10,           # 내부 버퍼 크기  
    slop=0.1                  # 허용 시간 차이 (초)  
)
```

- `queue_size`: 매칭 대기 중인 메시지 최대 개수
- `slop=0.1`: timestamp 차이가 100ms 이내인 메시지만 매칭

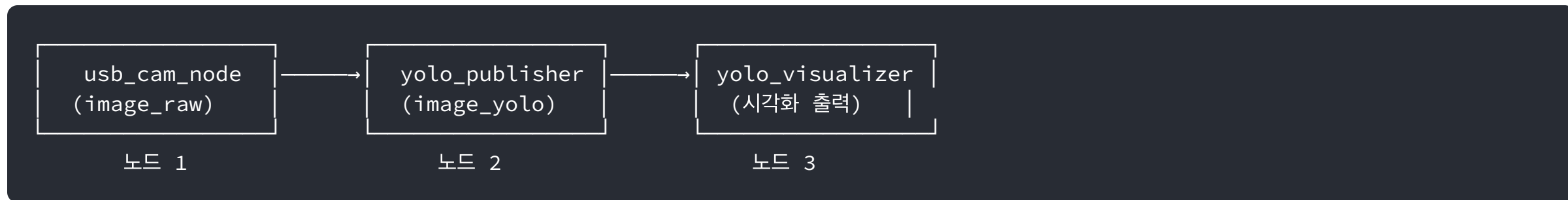
- ApproximateTimeSynchronizer vs TimeSynchronizer

동기화기	사용 시점	특징
TimeSynchronizer	timestamp가 정확히 일치 할 때	<code>header.stamp</code> 가 나노초 단위로 동일해야 매칭
ApproximateTimeSynchronizer	timestamp에 미세한 차이 가 있을 때	<code>slop</code> 파라미터로 허용 오차 조절, 실제 환경에서 더 유용

- 주의: `slop` 값 설정
 - 너무 작음 (0.01s): 매칭 실패 빈번 → 화면 갱신 끊김
 - 너무 큼 (1.0s): 전혀 다른 프레임이 매칭됨 → 시각적 불일치
 - **권장값**: 카메라 FPS의 1~2배 (30fps → 0.03~0.07s)



- 실행할 3개 노드



- launch 파일 역할

- 하나의 명령으로 3개 노드를 순차적/병렬적 실행